CHAPTER

5

Classification Problems

LEARNING OBJECTIVES

After completing this chapter, you will be able to

- Understand the concept of classification problems and their applications across different sectors.
- Learn to build logistic regression and decision tree models using the Python package *statsmodel* and *sklearn* APIs.
- Learn to perform activities such as
  (a) splitting the dataset into training and validation datasets.
  (b) building model using Python package on training dataset and test on the validation dataset.
- Learn to measure model accuracies using confusion matrix, sensitivity (recall), specificity, precision, receiver operating characteristic (ROC) curve, and area under ROC curve (AUC).
- Learn about statistical significance, model diagnostics and finding the optimal cut-off probabilities for classification.

5.1 | CLASSIFICATION OVERVIEW

Classification problems are an important category of problems in analytics in which the outcome variable or response variable (Y) takes discrete values. Primary objective of a classification model is to predict the probability of an observation belonging to a class, known as **class probability**. Few examples of classification problem are as follows:

1. A bank would like to classify the customers based on risk such as low-risk or high-risk customers.
2. E-commerce providers would like to predict whether a customer is likely to churn or not. It is a loss of revenue for the company if an existing and valuable customer churns.
3. Health service providers may classify a patient, based on the diagnostic results, as positive (presence of disease) or negative.
4. The HR department of a firm may want to predict if an applicant would accept an offer or not.
5. Predicting outcome of a sporting event, for example, whether India will win the next world cup cricket tournament.

6. Sentiments of customers on a product or service may be classified as positive, negative, neutral, and sarcastic.
7. Based on the image of a plant, one can predict if the plant is infected with a specific disease or not.

Classification problems may have binary or multiple outcomes or classes. Binary outcomes are called **binary classification** and multiple outcomes are called **multinomial classification**. There are several techniques used for solving classification problems such as logistic regression, classification trees, discriminant analysis, neural networks, and support vector machines. In this chapter, we will focus on *logistic regression* and *classification tree* (aka *decision tree learning*) for building and evaluating classification models. Some more models will be discussed in *Chapter 6*.

5.2 | BINARY LOGISTIC REGRESSION

Logistic regression is a statistical model in which the response variable takes a discrete value and the explanatory variables can either be continuous or discrete. If the outcome variable takes only two values, then the model is called binary logistic regression model. Assume that the outcomes are called positive (usually coded as $Y = 1$) and negative (usually coded as $Y = 0$). Then the probability that a record belongs to a positive class, $P(Y = 1)$, using the binary logistic regression model is given by

$$P(Y=1) = \frac{e^Z}{1+e^Z} \tag{5.1}$$

where

$$Z = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \cdots + \beta_m X_m$$

Here $X_1, X_2, \ldots, X_m$ are the independent variables or features. The logistic regression has an *S*-shaped curve, and gives the class probability of an observation belonging to class labelled as 1, that is, $P(Y = 1)$.

Equation (5.1) can be re-written as

$$\ln\left(\frac{P(Y=1)}{1-P(Y=1)}\right) = Z = \beta_0 + \beta_1 X_1 + \cdots + \beta_m X_m \tag{5.2}$$

The left-hand side of Eq. (5.2) is a log natural of odds and is known as logit function; the right-had side is a linear function. Such models are called **generalized linear models** (GLM). In GLM, the errors may not follow normal distribution and there exists a transformation function of the outcome variable that takes a linear functional form.

The logistic function in Eq. (5.1) has an *S*-shaped curve (thus also known as Sigmoid function). Consider an example of classifying a person with heart disease and without heart disease. That is, we are interested in finding the probability of a person suffering from heart disease as a function of age. The outcome variable is either someone suffers from heart disease or not and the independent variable (feature) is *age*. Then a logistic or sigmoid function can be fit to explain the probability of suffering from heart disease with respect to age as shown in Figure 5.1.

The corresponding logistic function is given as

$$P(\text{Heart Disease} = 1) = \frac{e^{\beta_0+\beta_1 Age}}{1+e^{\beta_0+\beta_1 Age}}$$

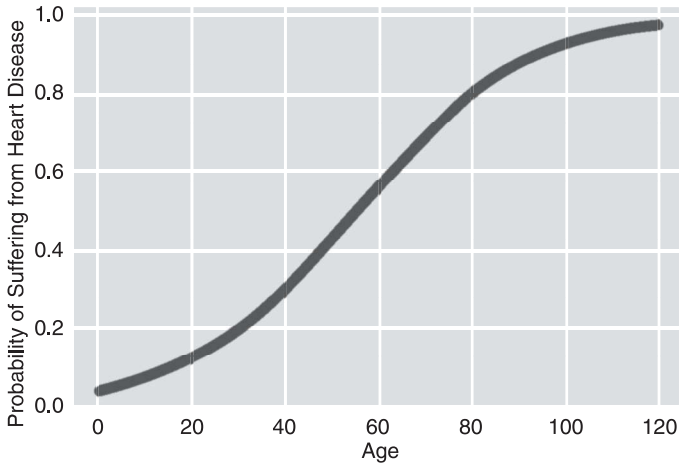


FIGURE 5.1 Logistic regression model for predicting probability of suffering from Heart Disease.

5.3 | CREDIT CLASSIFICATION

In this chapter, we will use the German credit rating dataset available at the University of California Irvine (UCI) machine learning laboratory to predict whether a credit is a good or bad credit. Credit classification is one of the popular classification problems encountered by financial institutions. For example, when a customer applies for a loan, financial institutions such as banks would like to predict the probability of default. In this section, we will be using the German credit rating dataset provided by Prof. Hofmann (available at UCI machine learning data repository[1]). The data contains several attributes (Table 5.1) of the persons who availed the credit.

TABLE 5.1 Data description for German credit rating dataset

Variable	Variable Type	Description	Categories
checkin_acc	categorical	Status of existing checking account	• A11 : … < 0 DM • A12 : 0 <= … < 200 DM • A13 : … >= 200 DM / salary assignments for at least 1 year • A14 : no checking account
duration	numerical	Duration of the credit given in months	
credit_history	categorical	Credit History	• A30 : no credits taken/all credits paid back duly • A31 : all credits at this bank paid back duly • A32 : existing credits paid back duly till now • A33 : delay in paying off in the past • A34 : critical account/other credits existing (not at this bank)

(*Continued*)

[1] Source: https://archive.ics.uci.edu/ml/datasets/Statlog+%28German+Credit+Data%29.

TABLE 5.1 (Continued)

Variable	Variable Type	Description	Categories
amount	numerical	Amount of credit/loan	
savings_acc	categorical	Balance in savings account	• A61 : … <100 DM • A62 : 100 <= … <500 DM • A63 : 500 <= … < 1000 DM • A64 : .. >=1000 DM • A65 : unknown/no savings account
present_emp_since	categorical	Employment in years	• A71 : unemployed • A72 : … < 1 year • A73 : 1 <=…< 4 years • A74 : 4 <=…< 7 years • A75 : .. >= 7 years
inst_rate	numerical	Installment rate	
personal_status	categorical	Marital status	• A91 : male : divorced/separated • A92 : female : divorced/separated/married • A93: male : single • A94 : male : married/widowed • A95 : female : single
residing_since	numerical	Residing since in years	
age	numerical	Age in years	
inst_plans	categorical	Other installment plans of the applicant	• A141 : bank • A142 : stores • A143 : none
checkin_acc	categorical	Balance in checking account	• A11 : … < 0 DM • A12 : 0 <= … < 200 DM • A13 : … >= 200 DM/salary assignments for at least 1 year • A14 : no checking account
job	categorical	Job	• A171 : unemployed/unskilled - non-resident • A172 : unskilled - resident • A173 : skilled employee/official • A174 : management/self-employed/highly qualified employee/officer
status	categorical	Credit status	• 0: Good Credit • 1: Bad Credit

The dataset contains records with comma separated values (*csv*) and can be read using pandas' *read_csv()* method.

Reading and displaying few records from the dataset is shown below:

```
import pandas as pd
import numpy as np

credit_df = pd.read_csv( "German Credit Data.csv" )
credit_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 14 columns):
checkin_acc          1000   non-null   object
duration             1000   non-null   int64
credit_history       1000   non-null   object
amount               1000   non-null   int64
savings_acc          1000   non-null   object
present_emp_since    1000   non-null   object
inst_rate            1000   non-null   int64
personal_status      1000   non-null   object
residing_since       1000   non-null   int64
age                  1000   non-null   int64
inst_plans           1000   non-null   object
num_credits          1000   non-null   int64
job                  1000   non-null   object
status               1000   non-null   int64
dtypes: int64(7), object(7)
memory usage: 109.5+ KB
```

The dataset contains 1000 records and 14 columns. Printing all the columns together will exceed the page width, so we will print the first 7 columns, that is, [1:7] and then print the remaining ones. The first column is a serial number and hence not selected for printing.

```
credit_df.iloc[0:5,1:7]
```

	duration	credit_history	Amount	savings_acc	present_emp_since	inst_rate
0	6	A34	1169	A65	A75	4
1	48	A32	5951	A61	A73	2
2	12	A34	2096	A61	A74	2
3	42	A32	7882	A61	A74	2
4	24	A33	4870	A61	A73	3

Now, let us print the remaining 7 columns.

```
credit_df.iloc[0:5,7:]
```

	personal_status	residing_since	age	inst_plans	num_credits	job	status
0	A93	4	67	A143	2	A173	0
1	A92	2	22	A143	1	A173	1
2	A93	3	49	A143	1	A172	0
3	A93	4	45	A143	1	A173	0
4	A93	4	53	A143	2	A173	1

There are few columns which are categorical and have been inferred as objects. For example, *checkin_acc* has following categories:

1. A11 : ... < 0 DM
2. A12 : 0 <= ... < 200 DM
3. A13 : ... >= 200 DM/salary assignments for at least 1 year
4. A14 : no checking account

To find the number of observations in the dataset for good credit and for bad credit, a simple *value_count()* on the *status* column will provide the information.

```
credit_df.status.value_counts()
```

```
0  700
1  300
Name: status, dtype: int64
```

The output displays that there are 700 observations of good credit and 300 observations of bad credit.

For building a classification model, the *status* column will be used as the dependent variable, while the remaining columns will be independent variables or features. We will create a list named *X_features* and store the names of all independent variables for future use.

```
X_features = list( credit_df.columns )
X_features.remove( 'status' )
X_features
```

```
['checkin_acc',
 'duration',
 'credit_history',
 'amount',
 'savings_acc',
 'present_emp_since',
 'inst_rate',
 'personal_status',
 'residing_since',
```

```
 'age',
 'inst_plans',
 'num_credits',
 'job']
```

5.3.1 | Encoding Categorical Features

There are several categorical features in the data which need to be binary encoded (using dummy variables), also known as one hot encoding (OHE). Details of encoding a categorical feature is already explained in Section 4.5.3, Chapter 4. Pandas method *pd.get_dummies()* is used to encode the categorical features and create dummy variables. If the feature has n categories, then either n or $(n - 1)$ dummy variables can be created [the model will have only $(n - 1)$ dummy variables]. If $(n - 1)$ categories are created, then the omitted category serves as base category. *pd.get_dummies()* takes a parameter *drop_first*, which drops the first category if set to *True*.

```
encoded_credit_df = pd.get_dummies(credit_df[X_features],
                                   drop_first = True)
```

We can list the new dummy features that have been created along with other continuous features.

```
list(encoded_credit_df.columns)
```

```
['duration',
 'amount',
 'inst_rate',
 'residing_since',
 'age',
 'num_credits',
 'checkin_acc_A12',
 'checkin_acc_A13',
 'checkin_acc_A14',
 'credit_history_A31',
 'credit_history_A32',
 'credit_history_A33',
 'credit_history_A34',
 'savings_acc_A62',
 'savings_acc_A63',
 'savings_acc_A64',
 'savings_acc_A65',
 'present_emp_since_A72',
 'present_emp_since_A73',
 'present_emp_since_A74',
 'present_emp_since_A75',
 'personal_status_A92',
 'personal_status_A93',
 'personal_status_A94',
```

```
'inst_plans_A142',
'inst_plans_A143',
'job_A172',
'job_A173',
'job_A174']
```

For example, *checkin_acc* variable is encoded into the following three dummy variables and the base category.

checkin_acc_A11 is dropped. If all the following three variables' values are set to 0, it indicates the account type is *checkin_acc_A11*.

1. checkin_acc_A12
2. checkin_acc_A13
3. checkin_acc_A14

Displaying the dummy features to verify how they are encoded.

```
encoded_credit_df[['checkin_acc_A12',
                   'checkin_acc_A13',
                   'checkin_acc_A14']].head(5)
```

	checkin_acc_A12	checkin_acc_A13	checkin_acc_A14
0	0	0	0
1	1	0	0
2	0	0	1
3	0	0	0
4	0	0	0

The first record has all the dummy variable values set to 0, indicating it is checkin account type *checkin_acc_A11*, which is the *base category*. The account type in second observation is *checkin_acc_A12*, so on.

As the variables, have been prepared, we can set *X (features)* and *Y (outcome)* variables before proceeding to build the model. Similar to *OLS()* method to build a linear regression model (as explained in *Chapter 4*), *statsmodel* api provides *Logit()* method for building logistic regression model. We need to add a new column and set its value to 1, for the model to estimate the intercept. In logistic regression, the regression parameters (feature weights) are estimated using maximum likelihood estimation.

```
import statsmodels.api as sm

Y = credit_df.status
X = sm.add_constant(encoded_credit_df)
```

5.3.2 | Splitting Dataset into Training and Test Sets

Before building the model, split the dataset into 70:30 (or 80:20) ratio for creating training and validation datasets. The model will be built using the training set and tested using test set.

```
from sklearn.model_selection import
X_train,X_test,y_train,y_test = train_test_split(X,
                                                 Y,
                                                 test_size = 0.3,
                                                 random_state = 42)
```

X_train and *y_train* contain the independent variables and response variable values for the training dataset respectively. Similarly, *X_test* and *y_test* contain the independent variables and response variable values for the test dataset, respectively.

5.3.3 | Building Logistic Regression Model

We will fit the model using *Logit* and pass training sets *y_train* and *X_train* as parameters. Invoking *fit()* method available in *statismodel.api* module helps to estimate the parameters for the logistic regression model.

```
import statsmodels.api as sm

logit = sm.Logit(y_train, X_train)
logit_model = logit.fit()
```

```
Optimization terminated successfully.
             Current function value: 0.488938
             Iterations 6
```

The method *fit()* on *Logit()* estimates the parameters and returns model details in the variable *logit_model*, which contains the model parameters, accuracy measures, residual values among other details.

5.3.4 | Printing Model Summary

The model object *logit_model* has *summary2()* method, which provides details of the model, its parameters and results of all necessary statistical tests. The model summary contains important information to validate the model.

```
logit_model.summary2()
```

Model:	Logit	Pseudo R-squared:	0.198
Dependent Variable:	status	AIC:	744.5132
Date:	2018-05-06 18:46	BIC:	881.0456
No. Observations:	700	Log-Likelihood:	–342.26

(*Continued*)

Df Model:	29	LL-Null:	–426.75
Df Residuals:	670	LLR p-value:	1.0630e-21
Converged:	1.0000	Scale:	1.0000
No. Iterations:	6.0000		

	Coef.	Std.Err.	*z*	*P* > |*z*|	[0.025	0.975]
const	–0.1511	1.1349	–0.1331	0.8941	–2.3754	2.0733
duration	0.0206	0.0104	1.9927	0.0463	0.0003	0.0409
amount	0.0001	0.0000	2.3765	0.0175	0.0000	0.0002
inst_rate	0.3064	0.0986	3.1083	0.0019	0.1132	0.4996
residing_since	0.0967	0.0920	1.0511	0.2932	–0.0836	0.2771
age	–0.0227	0.0103	–2.2131	0.0269	–0.0428	–0.0026
num_credits	0.2854	0.2139	1.3342	0.1821	–0.1338	0.7045
checkin_acc_A12	–0.4126	0.2391	–1.7260	0.0843	–0.8812	0.0559
checkin_acc_A13	–0.9053	0.4338	–2.0868	0.0369	–1.7556	–0.0550
checkin_acc_A14	–1.6052	0.2586	–6.2073	0.0000	–2.1120	–1.0983
credit_history_A31	0.1532	0.5795	0.2643	0.7916	–0.9827	1.2890
credit_history_A32	–0.4960	0.4411	–1.1245	0.2608	–1.3604	0.3685
credit_history_A33	–0.8881	0.5022	–1.7683	0.0770	–1.8724	0.0962
credit_history_A34	–1.4124	0.4528	–3.1190	0.0018	–2.2999	–0.5249
savings_acc_A62	–0.0496	0.3208	–0.1545	0.8772	–0.6782	0.5791
savings_acc_A63	–0.6640	0.4818	–1.3779	0.1682	–1.6084	0.2804
savings_acc_A64	–1.1099	0.6019	–1.8439	0.0652	–2.2896	0.0699
savings_acc_A65	–0.6061	0.2745	–2.2080	0.0272	–1.1441	–0.0681
present_emp_since_A72	0.0855	0.4722	0.1810	0.8564	–0.8401	1.0110
present_emp_since_A73	–0.0339	0.4492	–0.0754	0.9399	–0.9142	0.8465
present_emp_since_A74	–0.3789	0.4790	–0.7910	0.4289	–1.3178	0.5600
present_emp_since_A75	–0.2605	0.4554	–0.5721	0.5673	–1.1532	0.6321
personal_status_A92	–0.0069	0.4841	–0.0142	0.9887	–0.9557	0.9419
personal_status_A93	–0.4426	0.4764	–0.9291	0.3528	–1.3762	0.4911
personal_status_A94	–0.3080	0.5554	–0.5546	0.5792	–1.3967	0.7806
inst_plans_A142	–0.2976	0.5157	–0.5772	0.5638	–1.3084	0.7131
inst_plans_A143	–0.4458	0.2771	–1.6086	0.1077	–0.9889	0.0974
job_A172	–0.0955	0.7681	–0.1243	0.9011	–1.6009	1.4100
job_A173	–0.0198	0.7378	–0.0269	0.9786	–1.4658	1.4262
job_A174	–0.0428	0.7371	–0.0581	0.9537	–1.4876	1.4019

5.3.5 | Model Diagnostics

It is important to validate the logistic regression model to ensure its validity and goodness of fit before it can be used for practical applications. The following measures are used to validate a logistic regression model:

1. Wald's test (a Chi-square test) for checking the statistical significance of individual predictor (feature) variables. This is equivalent to *t*-test in the MLR (Multiple Linear Regression) model.
2. Likelihood ratio test for checking the statistical significance of the overall model (LLR *p*-value is reported in the output). Likelihood ratio test is also used for variable (feature) selection.
3. Pseudo R^2: It is a measure of goodness of the model. It is called pseudo R^2 because it does not have the same interpretation of R^2 as in the MLR model.

The model summary suggests that as per Wald's test, only 8 features are statistically significant at a significant value of $\alpha = 0.05$, as *p*-values are less than 0.05. *p*-value for likelihood ratio test (almost 0.00) indicates that the overall model is statistically significant.

Defining a method *get_significant_vars() that* takes the model object as an input and returns the list of significant variables after filtering out the variable with corresponding *p*-value less than 0.05.

```
def get_significant_vars( lm ):
    #Store the p-values and corresponding column names in a dataframe
    var_p_vals_df = pd.DataFrame( lm.pvalues )
    var_p_vals_df['vars'] = var_p_vals_df.index
    var_p_vals_df.columns = ['pvals', 'vars']
    # Filter the column names where p-value is less than 0.05
    return list( var_p_vals_df[var_p_vals_df.pvals <= 0.05]['vars'] )
```

Invoking the above method *get_significant_vars()* and passing the *logit_model*, we can get the list of significant variables.

```
significant_vars = get_significant_vars( logit_model )

significant_vars
```

```
['duration',
 'amount',
 'inst_rate',
 'age',
 'checkin_acc_A13',
 'checkin_acc_A14',
 'credit_history_A34',
 'savings_acc_A65']
```

Only the features *'duration', 'amount', 'inst_rate', 'age', 'checkin_acc_A13', 'checkin_acc_A14', 'credit_history_A34', 'savings_acc_A65'* have come out as significant variables. Now, we can build a logistic regression using only the significant variables.

```
final_logit = sm.Logit( y_train,
              sm.add_constant( X_train [significant_vars] ) ).fit()
```

```
Optimization terminated successfully.
             Current function value: 0.511350
             Iterations 6
```

```
final_logit.summary2()
```

Model:	Logit	Pseudo R-squared:	0.161
Dependent Variable:	status	AIC:	733.8898
Date:	2018-05-06 18:50	BIC:	774.8495
No. Observations:	700	Log-Likelihood:	–357.94
Df Model:	8	LL-Null:	–426.75
Df Residuals:	691	LLR p-value:	7.4185e-26
Converged:	1.0000	Scale:	1.0000
No. Iterations:	6.0000		

	Coef.	Std.Err.	*z*	*P* > |*z*|	[0.025	0.975]
const	–0.8969	0.4364	–2.0551	0.0399	–1.7523	–0.0415
duration	0.0197	0.0098	2.0033	0.0451	0.0004	0.0390
amount	0.0001	0.0000	2.3205	0.0203	0.0000	0.0002
inst_rate	0.2811	0.0929	3.0264	0.0025	0.0991	0.4632
age	–0.0216	0.0089	–2.4207	0.0155	–0.0392	–0.0041
checkin_acc_A13	–0.8038	0.4081	–1.9697	0.0489	–1.6037	–0.0040
checkin_acc_A14	–1.5452	0.2187	–7.0649	0.0000	–1.9738	–1.1165
credit_history_A34	–0.8781	0.2319	–3.7858	0.0002	–1.3327	–0.4235
savings_acc_A65	–0.5448	0.2581	–2.1108	0.0348	–1.0507	–0.0389

The negative sign in coefficient value indicates that as the value of this variable increases, the probability of being a bad credit decreases, that is, $P(Y_i = 1)$ decreases. A positive sign means that the probability of being a bad credit increases as the corresponding value of the variable increases. Some observations from the model output are:

1. The log of odds ratio or probability of being a bad credit increases as *duration*, *amount*, *inst_rate* increases. For example, one unit change in duration results in *0.019723* unit change in log of odds ratio [refer to UD Kumar (2017) for detailed discussion].
2. The probability of being a bad credit decreases as age increases. This means that older people tend to pay back their credits ontime compared to younger people.

5.3.6 | Predicting on Test Data

We will use the *final_logit* model and the significant features to predict the class probability. The *predict()* method will return the predicted probabilities for each observation in the test dataset. We will store the actual predicted class and predicted probabilities in a dataframe to compare and calculate various accuracy measures.

```
y_pred_df=pd.DataFrame({"actual": y_test,
                        "predicted_prob": final_logit.predict(
                        sm.add_constant( X_test[significant_vars]))})
```

We can print the predictions of few test samples randomly using the sample method of DataFrame.

```
y_pred_df.sample(10, random_state = 42)
```

	actual	predicted_prob
557	1	0.080493
798	0	0.076653
977	0	0.345979
136	0	0.249919
575	0	0.062264
544	0	0.040768
332	1	0.833093
917	1	0.370667
678	0	0.388392
363	0	0.088952

To understand how many observations the model has classified correctly and how many it has not, a cut-off probability needs to be assumed. Assume that the cut-off probability is 0.5; then all observations with predicted probability of more than 0.5 will be predicted as bad credits and rest all as good credits. We will explore techniques on how to find the optimal cut-off probability later in *Section 5.3.10: Finding Optimal Classification Cut-off*. For now, let us assume it to be 0.5.

Now iterate through predicted probability of each observation using *map()* and tag the observation as bad credit (1) if probability value is more than 0.5 or as good credit (0) otherwise.

```
y_pred_df['predicted'] = y_pred_df.predicted_prob.map(
                         lambda x: 1 if x > 0.5 else 0)

y_pred_df.sample(10, random_state = 42)
```

	actual	predicted_prob	predicted
557	1	0.080493	0
798	0	0.076653	0
977	0	0.345979	0
136	0	0.249919	0
575	0	0.062264	0
544	0	0.040768	0
332	1	0.833093	1
917	1	0.370667	0
678	0	0.388392	0
363	0	0.088952	0

The *actual* column in the DataFrame depicts the actual label of the credit in the test set, while predicted column depicts what the model has predicted by taking 0.5 as cut-off probability value. For observation 557, the model predicts very low probability (0.08) of being a bad credit whereas it is actually a bad credit. The model has wrongly classified this one. But the model predicts high probability (0.833) of being a bad credit for observation 332, which is actually a bad credit. The model correctly predicted the class in this case.

It can be noticed from *actual* and *predicted* columns that some classifications are correct and some are wrong. For better understanding of this confusion, we can build a confusion matrix, which is discussed next.

5.3.7 | Creating a Confusion Matrix

Confusion matrix (also known as error matrix or classification table) is a matrix formed by checking the actual values and predicted values of observations in the dataset. To create a confusion matrix, first import *matplotlib.pyplot* and *seaborn* libraries to render all plots and charts.

```
import matplotlib.pyplot as plt
import seaborn as sn
%matplotlib inline
```

sklearn.metric module provides a method to build a confusion matrix if actual and predicted values are passed as inputs parameters. For better understanding, this matrix can be plotted as a heatmap. *seaborn's heatmap* function can render the confusion matrix for easier interpretation. Define a method *draw_cm()* that takes actual and predicted values, creates the confusion matrix, and draws the heatmap.

```
from sklearn import metrics
def draw_cm( actual, predicted ):
    ## Cret
    cm = metrics.confusion_matrix( actual, predicted, [1,0] )
    sn.heatmap(cm, annot=True,    fmt='.2f',
               xticklabels = ["Bad credit", "Good Credit"],
               yticklabels = ["Bad credit", "Good Credit"] )
```

```
    plt.ylabel('True label')
    plt.xlabel('Predicted label')
    plt.show()
```

Invoke the method *draw_cm()* to plot the results of our model predictions.

```
draw_cm( y_pred_df.actual,
         y_pred_df.predicted )
```

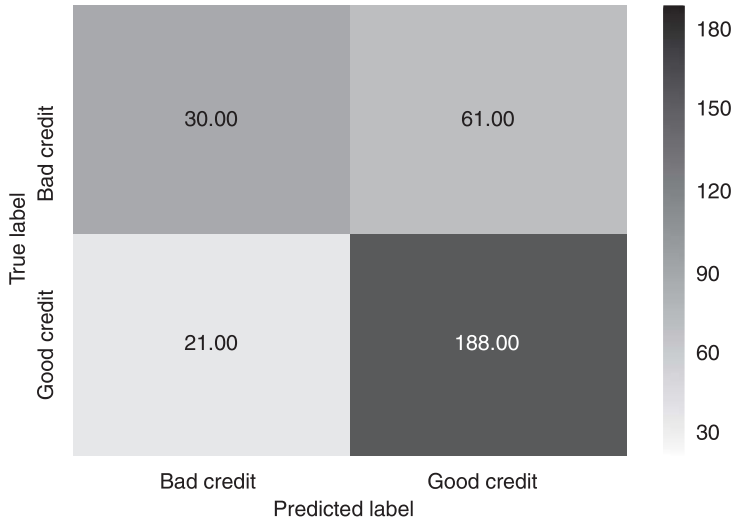


FIGURE 5.2 Confusion matrix with cut-off probability of 0.5.

In the confusion matrix in Figure 5.2, the columns represent the predicted label (class), while the rows represent the actual label (class). For example, out of 91 (i.e., 30 + 61) bad credits, only 30 have been classified correctly as bad credits and rest 61 have been classified as good credits when the cut-off probability is 0.5.

Each row represents the actual bad credit and good credit observations present in the test dataset and each column represents predicted values of outcome variable. We can note the following:

1. Left-top quadrant represents actual bad credit and is correctly classified as bad credit. This is called True Positives (TP).
2. Left-down quadrant represents actual good credit and is incorrectly classified as bad credit. This is called False Positives (FP).
3. Right-top quadrant represents actual bad credit and is incorrectly classified as good credit. This is called False Negatives (FN).
4. Right-down quadrant represents actual good credit and is correctly classified as good credit. This is called True Negatives (TN).

5.3.8 | Measuring Accuracies

In classification, the model performance is often measured using concepts such as sensitivity, specificity, precision, and F-score. The ability of the model to correctly classify positives and negatives is called **sensitivity** (also known as recall or true positive rate) and **specificity** (also known as true negative rate), respectively. The terminologies sensitivity and specificity originated in medical diagnostics.

Sensitivity or Recall (True Positive Rate)

Sensitivity is the conditional probability that the predicted class is positive given that the actual class is positive. Mathematically, sensitivity is given by

$$\text{Sensitivity} = \frac{TP}{TP + FN} \tag{5.3}$$

Specificity (True Negative Rate)

Specificity is the conditional probability that the predicted class is negative given that the actual class is negative. Mathematically, specificity is given by

$$\text{Specificity} = \frac{TN}{TN + FP} \tag{5.4}$$

Precision

Precision is the conditional probability that the actual value is positive given that the prediction by the model is positive. Mathematically, precision is given by

$$\text{Precision} = \frac{TP}{TP + FP} \tag{5.5}$$

F-Score

F-Score is a measure that combines precision and recall (harmonic mean between precision and recall). Mathematically, F-Score is given by

$$\text{F-Score} = \frac{2 \times \text{Recall} \times \text{Precision}}{\text{Recall} + \text{Precision}} \tag{5.6}$$

classification_report() method in *skearn.metrics* gives a detailed report of precision, recall, and F-score for each class.

```
print( metrics.classification_report( y_pred_df.actual,
                                      y_pred_df.predicted ) )
```

```
             precision    recall    f1-score    support
     0          0.76       0.90       0.82        209
     1          0.59       0.33       0.42         91
 avg/total      0.70       0.73       0.70        300
```

The model is very good at identifying the good credits ($Y = 0$), but not very good at identifying bad credits. This is the result for cut-off probability of 0.5%. This can be improved by choosing the right cut-off probability.

We can plot the distributions of predicted probability values for good and bad credits to understand how well the model can distinguish bad credits from good credits.

```
plt.figure( figsize = (8,6) )
# Plotting distribution of predicted probability values for bad credits
sn.distplot( y_pred_df[y_pred_df.actual == 1]["predicted_prob"],
             kde=False, color = 'b',
             label = 'Bad Credit')

# Plotting distribution of predicted probability values for good credits
sn.distplot( y_pred_df[y_pred_df.actual == 0]["predicted_prob"],
             kde=False, color = 'g',
             label = 'Good Credit')

plt.legend()
plt.show()
```

We can use a chart to understand how the distributions of predicted probabilities for bad credit and good credit look like (Figure 5.3). Larger the overlap between predicted probabilities for different classes, higher will be the misclassifications.

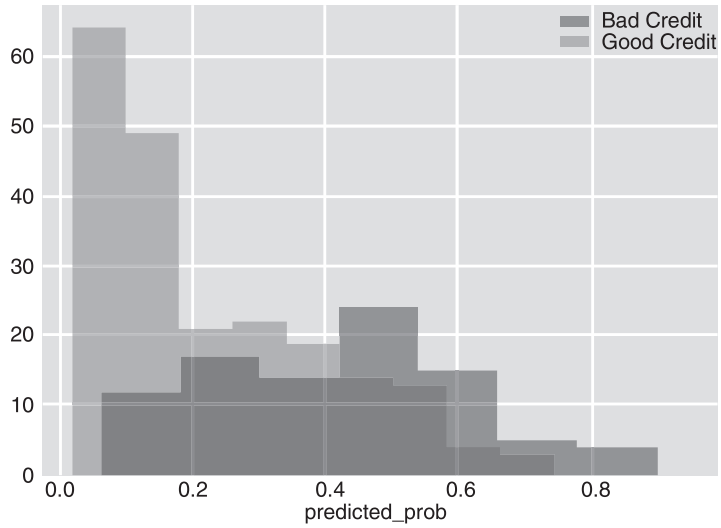


FIGURE 5.3 Distribution of predicted probability values by the model for both good and bad credits.

5.3.9 | Receiver Operating Characteristic (ROC) and Area Under the Curve (AUC)

Receiver operating characteristic (ROC) curve can be used to understand the overall performance (worth) of a logistic regression model (and, in general, of classification models) and used for model

selection. The term has its origin in electrical engineering when electrical signals were used for predicting enemy objects (such as submarines and aircraft) during World War II. Given a random pair of positive and negative class records, ROC gives the proportions of such pairs that will be correctly classified.

ROC curve is a plot between sensitivity (true positive rate) on the vertical axis and 1 – specificity (false positive rate) on the horizontal axis. We will write a method *draw_roc()* which takes the actual classes and predicted probability values and then draws the ROC curve (Figure 5.4).

metrics.roc_curve() returns different threshold (cut-off) values and their corresponding false positive and true positive rates. Then these values can be taken and plotted to create the ROC curve. *metrics.roc_auc_score()* returns the area under the curve (AUC).

```
def draw_roc( actual, probs ):
    # Obtain fpr, tpr, thresholds
    fpr,
    tpr,
    thresholds = metrics.roc_curve( actual,
                                    probs,
                                    drop_intermediate = False )
auc_score = metrics.roc_auc_score( actual, probs )
plt.figure(figsize=(8, 6))
# Plot the fpr and tpr values for different threshold values
plt.plot( fpr, tpr, label='ROC curve (area = %0.2f)' % auc_score )
# draw a diagonal line connecting the origin and top right most point
plt.plot([0, 1], [0, 1], 'k--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
# Setting x and y labels
plt.xlabel('False Positive Rate or [1 - True Negative Rate]')
plt.ylabel('True Positive Rate')
plt.legend(loc="lower right")
plt.show()

return fpr, tpr, thresholds
```

```
fpr, tpr, thresholds = draw_roc( y_pred_df.actual,
                                 y_pred_df.predicted_prob)
```

The diagonal line in Figure 5.4 represents the case of not using a model (no discrimination between positive and negative); the area below the diagonal line is equal to 0.5 (it is a right-angle triangle, area of right-angle triangle is 0.5*ab*, where *a* and *b* are the lengths of the sides which is equal to 1 in this case). Sensitivity and/or specificity are likely to change when the cut-off probability is changed. The line above the diagonal line in Figure 5.4 captures how sensitivity and (1 – specificity) change when the cut-off probability is changed. Model with higher AUC is preferred and AUC is frequently used for model selection.

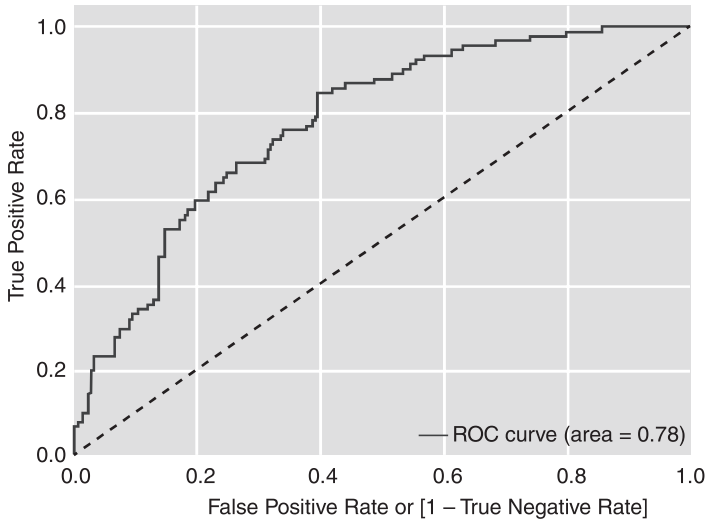


FIGURE 5.4 ROC curve.

As a thumb rule, AUC of at least 0.7 is required for practical application of the model. AUC greater than 0.9 implies an outstanding model. Caution should be exercised while selecting models based on AUC, especially when the data is imbalanced (i.e., dataset which has less than 10% positives). In case of imbalanced datasets, the AUC may be very high (greater than 0.9); however, either sensitivity or specificity values may be poor.

For this example, the AUC is 0.78, which implies the model is fairly good. The AUC can also be obtained by calling *roc_auc_score* from *sklearn.metrics*.

```
auc_score = metrics.roc_auc_score( y_pred_df.actual,
                                   y_pred_df.predicted_prob )
round( float( auc_score ), 2 )
```

```
0.78
```

5.3.10 | Finding Optimal Classification Cut-off

While using logistic regression model, one of the decisions that a data scientist has to make is to choose the right classification cut-off probability (P_c). The overall accuracy, sensitivity, and specificity will depend on the chosen cut-off probability. The following two methods are used for selecting the cut-off probability:

1. Youden's index
2. Cost-based approach

5.3.10.1 Youden's Index

Sensitivity and specificity change when we change the cut-off probability. Youden's index (Youden, 1950) is a classification cut-off probability for which the following function is maximized (also known as J-statistic):

$$\text{Youden's Index} = \text{J-Statistic} = \underset{p}{\text{Max}} \ [\text{Sensitivity}(p) + \text{Specificity}(p) - 1] \tag{5.7}$$

We already know that sensitivity is also known as True Positive Rate (TPR) and specificity is known as True Negative Rate (TNR). That is, select the cut-off probability for which (TPR + TNR – 1) is maximum.

draw_roc() has returned *tpr* and *fpr* values, which we have stored in variables *tpr, fpr*, respectively. The variable *thresholds* captures the corresponding cut-off probabilities. We can take difference of *tpr* and *fpr* and then sort the values in descending order. The *thresholds* value, for which Eq. (5.7) is maximum, should be the optimal cut-off.

```
tpr_fpr = pd.DataFrame( {  'tpr': tpr,
                           'fpr': fpr,
                           'thresholds': thresholds } )

tpr_fpr['diff'] = tpr_fpr.tpr - tpr_fpr.fpr
tpr_fpr.sort_values( 'diff', ascending = False )[0:5]
```

	fpr	thresholds	tpr	diff
159	0.397129	0.221534	0.846154	0.449025
160	0.401914	0.216531	0.846154	0.444240
161	0.406699	0.215591	0.846154	0.439455
158	0.397129	0.223980	0.835165	0.438036
165	0.421053	0.207107	0.857143	0.436090

From the above result, the optimal cut-off is 0.22. We can now classify all the observations beyond 0.22 predicted probability as bad credits and others as good credits. We will capture these new classes in *predicted_new* variable and then draw a new confusion matrix (Figure 5.5).

```
y_pred_df['predicted_new'] = y_pred_df.predicted_prob.map(
                             lambda x: 1 if x > 0.22 else 0)
```

```
draw_cm( y_pred_df.actual,
         y_pred_df.predicted_new)
```

Now print the report with cut-off probability of 0.22.

```
print(metrics.classification_report( y_pred_df.actual,
                                     y_pred_df.predicted_new ))
```

```
             precision    recall  f1-score   support

          0       0.90      0.60      0.72       209
          1       0.48      0.85      0.61        91
avg/total       0.77      0.68      0.69       300
```

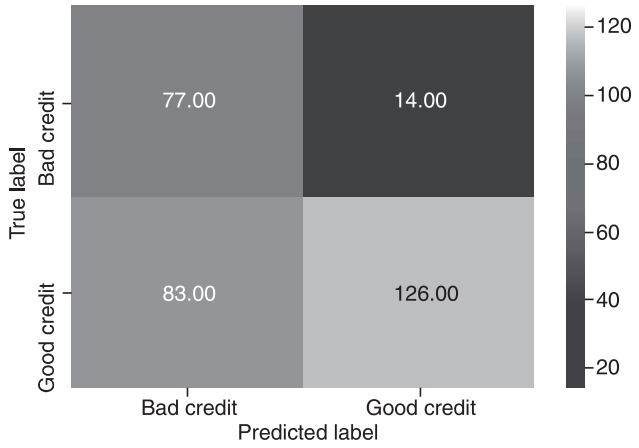


FIGURE 5.5 Confusion matrix with optimal cut-off using Yoden's index.

With cut-off probability of 0.22, the model is able to classify the bad credits better and the F1-score for bad credits ($Y = 1$) has also improved to 0.61.

5.3.10.2 Cost-Based Approach

As the cost of false negatives and false positives is not same, the optimal classification cut-off probability can also be determined using cost-based approach, which finds the cut-off where the total cost is minimum. In the cost-based approach, we assign penalty cost for misclassification of positives and negatives and find the total cost for a cut-off probability.

Assuming cost of a false positive is C1 and that of a false negative is C2, total cost will be

$$\text{Total cost} = \text{FN} \times \text{C1} + \text{FP} \times \text{C2}$$

The optimal cut-off probability is the one which minimizes the total penalty cost. We will write a method *get_total_cost()* to return the penalty cost for a specific cut-off probability. The method also takes the relative cost of false negatives and false positives. The function will take actual and predicted values, find the number of false positives (FPs) and false negatives (FNs), and return the total cost.

```
def get_total_cost( actual, predicted, cost_FPs, cost_FNs ):
    # Get the confusion matrix and calculate cost
    cm = metrics.confusion_matrix( actual, predicted, [1,0] )
    cm_mat = np.array( cm )
    return cm_mat[0,1] * cost_FNs + cm_mat[1,0] * cost_FPs
```

Create a DataFrame which will capture the cost against different cut-off probability values.

```
cost_df = pd.DataFrame( columns = ['prob', 'cost'])
```

Let us assume that false negatives (predicting a bad credit to be a good credit) are five times costlier than false positives (predicting a good credit to be a bad credit). Deciding the costs will require domain knowledge.

We can calculate the penalty cost for each cut-off probability values between 0.1 and 0.5 with incremental values of 0.01 and sort the costs in ascending order to find the cut-off probability at which the penalty cost is minimum.

```
idx = 0

## Iterate cut-off probability values between 0.1 and 0.5
for each_prob in range(10, 50):
    cost = get_total_cost( y_pred_df.actual,
                           y_pred_df.predicted_prob.map(
           lambda x: 1 if x > (each_prob/100)      else 0), 1, 5 )
    cost_df.loc[idx] = [(each_prob/100), cost]
    idx += 1
```

```
cost_df.sort_values( 'cost', ascending = True )[0:5]
```

	Prob	Cost
4	0.14	150.0
12	0.22	153.0
2	0.12	154.0
10	0.20	154.0
9	0.19	156.0

The lowest cost is achieved at cut-off probability of 0.14 if false negatives are assumed to be five times costlier than false positives. So, let us predict everything beyond 0.14 as bad credit and below 0.14 as good credit.

```
y_pred_df['predicted_using_cost'] = y_pred_df.predicted_prob.map(
  lambda x: 1 if x > 0.14 else 0)
```

```
draw_cm( y_pred_df.actual,
         y_pred_df.predicted_using_cost )
```

As shown in Figure 5.6, the model has reduced the false negatives to only 6. This is because of high cost false negatives (5 times) compared to false positives. The number of false positives have increased to 120.

5.4 | GAIN CHART AND LIFT CHART

Gain chart and lift chart are two measures that are used for measuring the benefits of using the logistic regression model (in general, analytical model) and are used in business contexts such as target marketing. In target marketing or marketing campaigns, customers' responses to campaign are usually very low (in many cases the customers who respond to marketing campaigns are less than 1%). Another example of such low conversion is response to advertisement on the Internet (such as Google Adwords

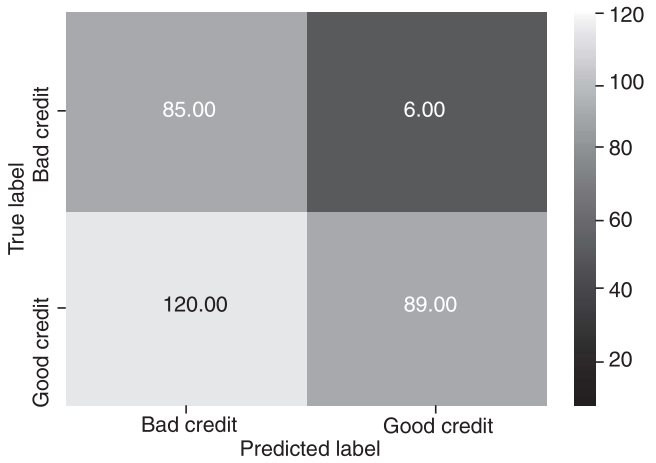


FIGURE 5.6 Confusion matrix with cost-based optimal cut-off.

and mobile advertisements). The organization incurs cost for each customer contact and hence would like to minimize the cost of marketing campaign and at the same time achieve the desired response level from the customers. The gain and lift charts are obtained using the following steps:

1. Predict the probability $Y = 1$ (positive) using the logistic regression (LR) model and arrange the observation in the decreasing order of predicted probability [i.e., $P(Y = 1)$].
2. Divide the datasets into deciles. Calculate the number of positives ($Y = 1$) in each decile and cumulative number of positives up to a decile.
3. Gain is the ratio between cumulative number of the positive observations up to a decile to total number of positive observations in the data. Gain chart is a chart drawn between gain on the vertical axis and decile on the horizontal axis.
4. Lift is the ratio of the number of positive observations up to decile i using the LR model to the expected number of positives up to that decile i based on a random model (not using a model). Lift chart is the chart between lift on the vertical axis and the corresponding decile on the horizontal axis.

$$\text{Gain} = \frac{\text{Cumulative number of positive observations upto decile } i}{\text{Total number of positive observations in the data}}$$

$$\text{Lift} = \frac{\text{Cumulative number of positive observations upto decile } i \text{ using LR model}}{\text{Cumulative number of positive observations upto decile } i \text{ based on random model}}$$

To illustrate the gain and lift charts, we will be using the *bank marketing dataset* available at the University of California, Irvine (UCI) machine learning repository[2]. The data describes a problem in which a bank is interested in predicting which customers may respond to their direct marketing campaign to open a term deposit with the bank. The response variable $Y = 1$ implies that the customer opens a term deposit after the campaign and $Y = 0$ otherwise. The marketing campaign is based on the phone calls.

We will use the smaller dataset *bank.csv*. The description of the features is provided in Table 5.2.

[2] The dataset is taken from https://archive.ics.uci.edu/ml/datasets/bank+marketing (https://archive.ics.uci.edu/ml/datasets/bank+marketing). The description of all the variables are also available at the same location.

TABLE 5.2 Data description for bank marketing dataset

Variable	Variable Type	Description
age	numeric	Age of the client who is the targe of this marketing exercise
job	categorical	type of job (categories: 'admin.', 'blue-collar', 'entrepreneur', 'housemaid', 'management', 'retired', 'self- employed', 'services', 'student', 'technician', 'unemployed', 'unknown')
marital	categorical	marital status (categories: "married", "divorced", "single"; note: "divorced" means divorced or widowed)
education	categorical	education qualification (categories: "unknown", "secondary", "primary", "tertiary")
default	categorical	customer has credit in default? (categories: 'no', 'yes')
balance	numerical	average yearly balance, in euros
housing-loan	categorical	has housing loan? (categories: 'no', 'yes')
personal-loan	categorical	has personal loan? (categories: 'no', 'yes')
previous-campaign	numerical	number of contacts performed before this campaign and for this client
subscribed	categorical	has the client subscribed a term deposit? (binary: "yes", "no")

5.4.1 | Loading and Preparing the Dataset

The dataset *bank.csv* contains records with comma separated values and can be read using pandas' *read_csv()* method.

Using the following code, we can read and display the first 5 records.

```
import pandas as pd
bank_df = pd.read_csv( 'bank.csv' )
bank_df.head( 5 )
```

	age	job	marital	education	default	balance	housing-loan	personal-loan	current-campaign	previous-campaign	subscribed
0	30	unemployed	married	primary	no	1787	no	no	1	0	no
1	33	services	married	secondary	no	4789	yes	yes	1	4	no
2	35	management	single	tertiary	no	1350	yes	no	1	1	no
3	30	management	married	tertiary	no	1476	yes	yes	4	0	no
4	59	blue-collar	married	secondary	no	0	yes	no	1	0	no

```
bank_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4521 entries, 0 to 4520
Data columns (total 11 columns):

Age                 4521    non-null    int64
Job                 4521    non-null    object
```

```
Marital              4521   non-null   object
Education            4521   non-null   object
Default              4521   non-null   object
Balance              4521   non-null   int64
housing-loan         4521   non-null   object
personal-loan        4521   non-null   object
current-campaign     4521   non-null   int64
previous-campaign    4521   non-null   int64
Subscribed           4521   non-null   object
dtypes: int64(4), object(7)
memory usage: 388.6+ KB
```

The dataset has a total of 4521 observations, out of which 521 customers subscribed the term deposit (approximately 11.5%) and the remaining 4000 did not subscribe the term deposit. Except *age, balance, current-campaign,* and *previous-campaign*, rest all are (object) categorical features.

Let us capture the independent variables into the list *X_features*.

```
X_features = list( bank_df.columns )
X_features.remove( 'subscribed' )
X_features
```

```
['age',
 'job',
 'marital',
 'education',
 'default',
 'balance',
 'housing-loan',
 'personal-loan',
 'current-campaign',
 'previous-campaign']
```

Encode the categorical features into dummy variables using the following code:

```
encoded_bank_df = pd.get_dummies( bank_df[X_features],
                                  drop_first = True )
```

The outcome variable *subscribed* is set to *yes* or *no*. This needs to be encoded as 1 (yes) and 0 (no).

```
Y = bank_df.subscribed.map( lambda x: int( x == 'yes') )
X = encoded_bank_df
```

Here the dataset is not split into training and test datasets for simplicity as our objective is primarily to understand gain and lift chart.

5.4.2 | Building the Logistic Regression Model

We wil start with building a logistics regression model to predict probability of customer subscribing to term deposits using the features available in *X*.

```
logit_model = sm.Logit( Y, sm.add_constant( X ) ).fit()
```

```
Optimization terminated successfully.
             Current function value: 0.335572
             Iterations 7
```

```
logit_model.summary2()
```

Model:	Logit	Pseudo R-squared:	0.061
Dependent Variable:	subscribed	AIC:	3082.2384
Date:	2018-02-26 21:42	BIC:	3236.2341
No. Observations:	4521	Log-Likelihood:	−1517.1
Df Model:	23	LL-Null:	−1615.5
Df Residuals:	4497	LLR p-value:	1.4866e-29
Converged:	1.0000	Scale:	1.0000
No. Iterations:	7.0000		

	Coef.	Std.Err.	*z*	*P* > |*z*|	[0.025	0.975]
const	−1.7573	0.3799	−4.6251	0.0000	−2.5019	−1.0126
age	0.0078	0.0058	1.3395	0.1804	−0.0036	0.0191
balance	−0.0000	0.0000	−0.2236	0.8231	−0.0000	0.0000
current-campaign	−0.0905	0.0238	−3.8042	0.0001	−0.1371	−0.0439
previous-campaign	0.1414	0.0212	6.6569	0.0000	0.0998	0.1830
job_blue-collar	−0.3412	0.2000	−1.7060	0.0880	−0.7331	0.0508
job_entrepreneur	−0.2900	0.3161	−0.9175	0.3589	−0.9096	0.3295
job_housemaid	−0.0166	0.3339	−0.0497	0.9603	−0.6711	0.6379
job_management	−0.0487	0.1984	−0.2455	0.8061	−0.4375	0.3401
job_retired	0.5454	0.2503	2.1794	0.0293	0.0549	1.0360
job_self-employed	−0.2234	0.2895	−0.7715	0.4404	−0.7909	0.3441
job_services	−0.2248	0.2245	−1.0012	0.3167	−0.6648	0.2152
job_student	0.3888	0.3181	1.2223	0.2216	−0.2346	1.0122

(*Continued*)

	Coef.	Std.Err.	z	P > |z|	[0.025	0.975]
job_technician	–0.2101	0.1874	–1.1213	0.2622	–0.5773	0.1571
job_unemployed	–0.3723	0.3336	–1.1162	0.2643	–1.0261	0.2815
job_unknown	0.3193	0.4620	0.6913	0.4894	–0.5861	1.2248
marital_married	–0.4012	0.1440	–2.7857	0.0053	–0.6835	–0.1189
marital_single	–0.0463	0.1676	–0.2763	0.7823	–0.3749	0.2822
education_secondary	0.2128	0.1680	1.2670	0.2052	–0.1164	0.5420
education_tertiary	0.3891	0.1935	2.0103	0.0444	0.0098	0.7684
education_unknown	–0.1956	0.2927	–0.6682	0.5040	–0.7693	0.3781
default_yes	0.2286	0.3670	0.6228	0.5334	–0.4908	0.9479
housing-loan_yes	–0.5355	0.1024	–5.2273	0.0000	–0.7362	–0.3347
personal-loan_yes	–0.7139	0.1689	–4.2268	0.0000	–1.0449	–0.3829

Find out which are significant variables using the method *get_significant_vars()* defined in *Section 5.3.5: Model Diagnostics* and build a model with only the significant variables.

```
significant_vars = get_significant_vars(logit_model)

significant_vars
```

```
['const',
 'current-campaign',
 'previous-campaign',
 'job_retired',
 'marital_married',
 'education_tertiary',
 'housing-loan_yes',
 'personal-loan_yes']
```

Setting *X_features* to only significant variables and building a logistic regression model with the significant features.

```
X_features = ['current-campaign',
              'previous-campaign',
              'job_retired',
              'marital_married',
              'education_tertiary',
              'housing-loan_yes',
              'personal-loan_yes']
```

```
logit_model_2 = sm.Logit( Y, sm.add_constant( X[X_features] ) ).fit()
```

```
Optimization terminated successfully.
             Current function value: 0.337228
             Iterations 7
```

Printing the model summary.

```
logit_model_2.summary2()
```

Model:	Logit	Pseudo R-squared:	0.056
Dependent Variable:	subscribed	AIC:	3065.2182
Date:	2018-02-26 21:42	BIC:	3116.5501
No. Observations:	4521	Log-Likelihood:	−1524.6
Df Model:	7	LL-Null:	−1615.5
Df Residuals:	4513	LLR p-value:	8.1892e-36
Converged:	1.0000	Scale:	1.0000
No. Iterations:	7.0000		

	Coef.	Std.Err.	*z*	*P* > |*z*|	[0.025	0.975]
const	−1.4754	0.1133	−13.0260	0.0000	−1.6974	−1.2534
current-campaign	−0.0893	0.0236	−3.7925	0.0001	−0.1355	−0.0432
previous-campaign	0.1419	0.0211	6.7097	0.0000	0.1004	0.1833
job_retired	0.8246	0.1731	4.7628	0.0000	0.4853	1.1639
marital_married	−0.3767	0.0969	−3.8878	0.0001	−0.5667	−0.1868
education_tertiary	0.2991	0.1014	2.9500	0.0032	0.1004	0.4978
housing-loan_yes	−0.5834	0.0986	−5.9179	0.0000	−0.7767	−0.3902
personal-loan_yes	−0.7025	0.1672	−4.2012	0.0000	−1.0302	−0.3748

p_value for LLR (Likelihood Ratio test) shows (less than 0.05) that the overall model is significant. We will predict the probabilities of the same observations as we have not split the dataset. But if readers have split, then they can use the test dataset for predicting the probabilities.

```
y_pred_df = pd.DataFrame( {  'actual': Y,
                            'predicted_prob': logit_model_2.predict(
                            sm.add_constant( X[X_features] ) ) } )
```

Now sort the observations by their predicted probabilities in the descending order.

```
sorted_predict_df = y_pred_df[['predicted_prob',
                               'actual']].sort_values('predicted_prob',
                               ascending = False)
```

After sorting, we will segment all the observations into deciles. First we will find the number of observations in each decile by dividing the total number of observations by 10.

```
num_per_decile = int( len( sorted_predict_df ) / 10 )
print("Number of observations per decile:", num_per_decile)
```

```
Number of observations per decile:      452
```

The function *get_deciles()* takes a DataFrame and segments the observations into deciles and marks each observation with the decile number it belongs to. The DataFrame with sorted probabilities should be passed to this function.

```
def get_deciles(df):
    # Set first decile
    df['decile'] = 1

    idx = 0
    # Iterate through all 10 deciles
    for each_d in range(0, 10):
        # Setting each 452 observations to one decile in sequence
        df.iloc[idx:idx+num_per_decile, df.columns.get_
            loc('decile')] = each_d
        idx += num_per_decile

    df['decile'] = df['decile'] + 1

    return df
```

Now invoke the above method with *sorted_predict_df* as a parameter.

```
deciles_predict_df = get_deciles( sorted_predict_df )
```

The following code displays the first 10 observations in the DataFrame *sorted_predict_df*.

```
deciles_predict_df[0:10]
```

	predicted_prob	actual	decile
3682	0.864769	0	1
97	0.828031	0	1
3426	0.706809	0	1
1312	0.642337	1	1
3930	0.631032	1	1
4397	0.619146	0	1

	predicted_prob	actual	decile
2070	0.609129	0	1
3023	0.573199	0	1
4080	0.572364	0	1
804	0.559350	0	1

Calculating Gain

To calculate the gain, we need to find how many subscriptions (how many 1's) are available in each decile. For this, the *actual* column value can be summed for each decile.

```
gain_lift_df = pd.DataFrame(
     deciles_predict_df.groupby(
             'decile')['actual'].sum() ).reset_index()
gain_lift_df.columns = ['decile', 'gain']
```

And then the cumulative sum for each subsequent decile divided by the total number of 1's available will give the *gain percentage*.

```
gain_lift_df['gain_percentage'] = (100 * gain_lift_df.gain.
                                   cumsum()/gain_lift_df.gain.sum())
```

```
gain_lift_df
```

	decile	gain	gain_percentage
0	1	125	23.992322
1	2	83	39.923225
2	3	73	53.934741
3	4	53	64.107486
4	5	31	70.057582
5	6	46	78.886756
6	7	37	85.988484
7	8	28	91.362764
8	9	25	96.161228
9	10	20	100.000000

So, if we target only the first 50% of the customers, we have almost 70% subscriptions. The change in gain percentage can be plotted for better understanding (see Figure 5.7).

```
import matplotlib.pyplot as plt
import seaborn as sn
%matplotlib inline
```

```
plt.figure( figsize = (8,4))
plt.plot( gain_lift_df['decile'],
          gain_lift_df['gain_percentage'], '-' )

plt.show()
```

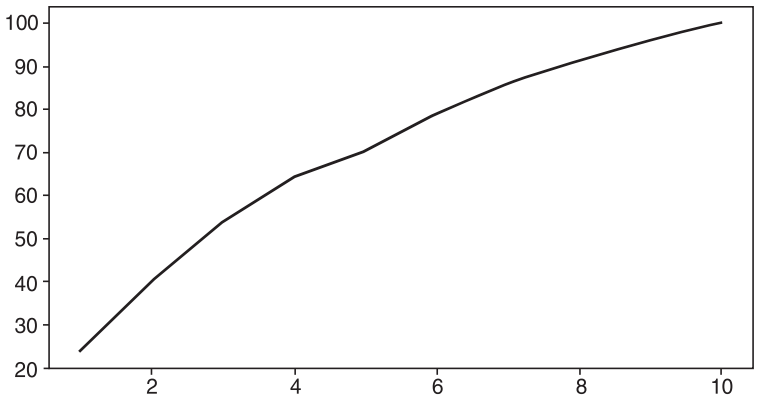


FIGURE 5.7 Gain chart.

The chart in Figure 5.7 shows the increase in gain against deciles. It can be observed that the marginal increase in gain reduces significantly as deciles progress.

Calculating Lift

Lift can be calculated by dividing gain percentage by the decile number.

```
gain_lift_df['lift'] = ( gain_lift_df.gain_percentage
                       / ( gain_lift_df.decile * 10 ) )
gain_lift_df
```

	decile	gain	gain_percentage	lift
0	1	125	23.992322	2.399232
1	2	83	39.923225	1.996161

(*Continued*)

	decile	gain	gain_percentage	lift
2	3	73	53.934741	1.797825
3	4	53	64.107486	1.602687
4	5	31	70.057582	1.401152
5	6	46	78.886756	1.314779
6	7	37	85.988484	1.228407
7	8	28	91.362764	1.142035
8	9	25	96.161228	1.068458
9	10	20	100.000000	1.000000

The *gain_lift_df* depicts the lift value against decile numbers.

```
plt.figure( figsize = (8,4))
plt.plot( gain_lift_df['decile'], gain_lift_df['lift'], '-' )
plt.show()
```

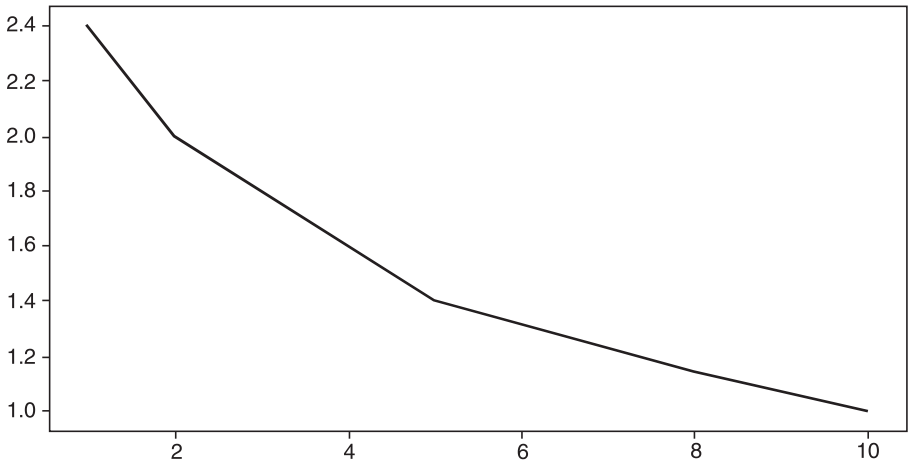


FIGURE 5.8 Lift chart.

As show in Figure 5.8 per the lift values, targeting customers using the model can capture 2.39 times the number of subscribers compared to a random model in decile 1, 1.99 times up to decile 2, and so on and so forth.

5.5 | CLASSIFICATION TREE (DECISION TREE LEARNING)

Decision Tree Learning or Classification Trees are a collection of divide and conquer problem-solving strategies that use tree-like (inverted tree in which the root is at the top) structure to predict the value of an outcome variable. The tree starts with the root node consisting of the complete data and thereafter

uses intelligent strategies to split the nodes (parent node) into multiple branches (thus, creating children nodes). The original data is divided into subsets in this process. This is done to create more homogenous groups at the children nodes. It is one of the most powerful predictive analytics techniques used for generating business rules.

Classification and Regression Tree (CART) is one of the classification tree techniques. CART is an umbrella term; we will call it a classification tree if the outcome variable value is discrete and a regression tree if the outcome variable value is continuous.

Classification tree uses various impurity measures such as the *Gini Impurity Index* and *Entropy* to split the nodes. Regression tree, on the other hand, splits the node that minimizes the *Sum of Squared Errors* (SSE).

The following steps are used to generate classification and regression trees:

1. Start with the complete training data in the root node.
2. Decide on the measure of impurity, that is, either Gini Impurity Index or Entropy. Search for a predictor variable that minimizes the impurity when the parent node is split into children nodes. This happens when the original data is divided into two subsets using a predictor variable such that it results in the maximum reduction in the impurity in the case of a discrete dependent variable or maximum reduction in SSE in the case of a continuous dependent variable.
3. Repeat step 2 for each subset of the data (for each internal node) using the independent variables until
   (a) All the dependent variables are exhausted.
   (b) The stopping criteria are met. Few stopping criteria used are number of levels of tree from the root node, minimum number of observations in parent/child node (e.g., 10% of the training data), and minimum reduction in impurity index.
4. Generate business rules for the leaf (terminal) nodes of the tree.

sklearn.tree.DecisionTreeClassifier provides decision tree algorithm to create the decision tree. It takes the following key parameters:

1. criterion: string – The function to measure the quality of a split. Supported criteria are "gini" for the Gini impurity and "entropy" for the information gain. Default is *gini*.
2. max_depth: int – The maximum depth of the tree.
3. min_samples_split: int or float – The minimum number of samples required to split an internal node. If *int*, then number of samples or if *float,* percentage of total number of samples. Default is 2.
4. in_samples_leaf: int or float – The minimum number of samples required to be at a leaf node.

Detailed documentation is available at *http://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html*

Let us build the model on the credit rating dataset.

5.5.1 | Splitting the Dataset

Set *encoded_credit_df* as *X* and the column *status* as *Y* and split the data into train and test sets using 70:30 splitting criteria.

```
Y = credit_df.status
X = encoded_credit_df
```

```
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test = train_test_split(X, Y,
                                                 test_size = 0.3,
                                                 random_state = 42 )
```

5.5.2 | Building Decision Tree Classifier using Gini Criteria

We have already read the dataset using *credit_df* variable. We will set *criterion* to *gini*, *max_depth* to 3, and keep default values for other parameters. In *Section 5.5.6: Finding Optimal Criteria and max_depth,* we will explore the most optimal criteria and max_depth.

```
from sklearn.tree import DecisionTreeClassifier
clf_tree = DecisionTreeClassifier(criterion = 'gini',
                                  max_depth = 3 )
```

```
clf_tree.fit( X_train, y_train )
```

```
DecisionTreeClassifier(class_weight=None, criterion='gini', max_
     depth=3,
     max_features=None, max_leaf_nodes=None,
     min_impurity_decrease=0.0, min_impurity_split=None,
     min_samples_leaf=1, min_samples_split=2,
     min_weight_fraction_leaf=0.0, presort=False, random_state=None,
     splitter='best')
```

5.5.3 | Measuring Test Accuracy

Using the model to predict the probability of a bad credit on the test set and obtain ROC AUC score.

```
tree_predict = clf_tree.predict( X_test )
metrics.roc_auc_score( y_test, tree_predict )
```

```
0.58
```

5.5.4 | Displaying the Tree

To visualize the tree, use *graphviz* software. For this the following software need to be installed on your machine.

1. GraphViz (as per the OS and version you are using)
2. pip install pydotplus

Then export the tree model to a file using *export_graphviz()* function.

```
from sklearn.tree import export_graphviz
import pydotplus as pdot
from IPython.display import Image

# Export the tree into odt file
export_graphviz( clf_tree,
                 out_file = "chd_tree.odt",
                 feature_names = X_train.columns,
                 filled = True )

# Read the create the image file
chd_tree_graph = pdot.graphviz.graph_from_dot_file( 'chd_tree.odt' )
chd_tree_graph.write_jpg( 'chd_tree.png' )
# Render the png file
Image(filename='chd_tree.png')
```

From the decision tree diagram in Figure 5.9, it can be interpreted that

1. At the top node, there are 700 observations of which 491 are good credits and 209 are bad credits. The corresponding *Gini* index is 0.419. Later in the section, the *Gini* impurity index is explained.
2. *checkin_account_A14* is the most important feature for splitting good and bad credits in the dataset when compared to other features and hence, chosen as the top splitting criteria.
3. The first rule (*checkin_account_A14* < 0.5) means if the customer has *checkin_account_A14* account or not.
4. This rule has split the dataset into two subsets represented by the second level nodes. On the left node, there are 425 samples (i.e., not having *checkin_account_A14*) and on the right node, there are 275 samples (i.e. having *checkin_account_A14*).
5. The nodes represented by dark shades depict good credits, while the nodes represented by light shades are bad credits.
6. One of the rules can be interpreted as: If the customer does not have *checkin_account_A14* and credit *duration* is less than 33 and does not have *saving_acc_A65*, then there is high probability of being a bad credit. There are 70 records in the dataset that satisfy these conditions and 48 of them have bad credit.
7. Another rule: If the customer has *checkin_account_A14* and *Inst_plans_A143* and age is more than 23.5, then there is a high probability of being good credit.

5.5.5 | Understanding Gini Impurity

Gini measurement is the probability of a random sample being classified correctly if we randomly pick a label according to the distribution in a branch. Gini impurity can be computed by summing the probability p_i of an item with label i being chosen times the probability $1 - p_i$ of a mistake in categorizing that item over all classes. It reaches its minimum (zero) when all cases in the node belong to a specific category. The Gini impurity index for a classification problem with C classes is given by

$$Gini(k) = \sum_{i=1}^{C} p_i(k)(1 - p_i(k)) \tag{5.8}$$

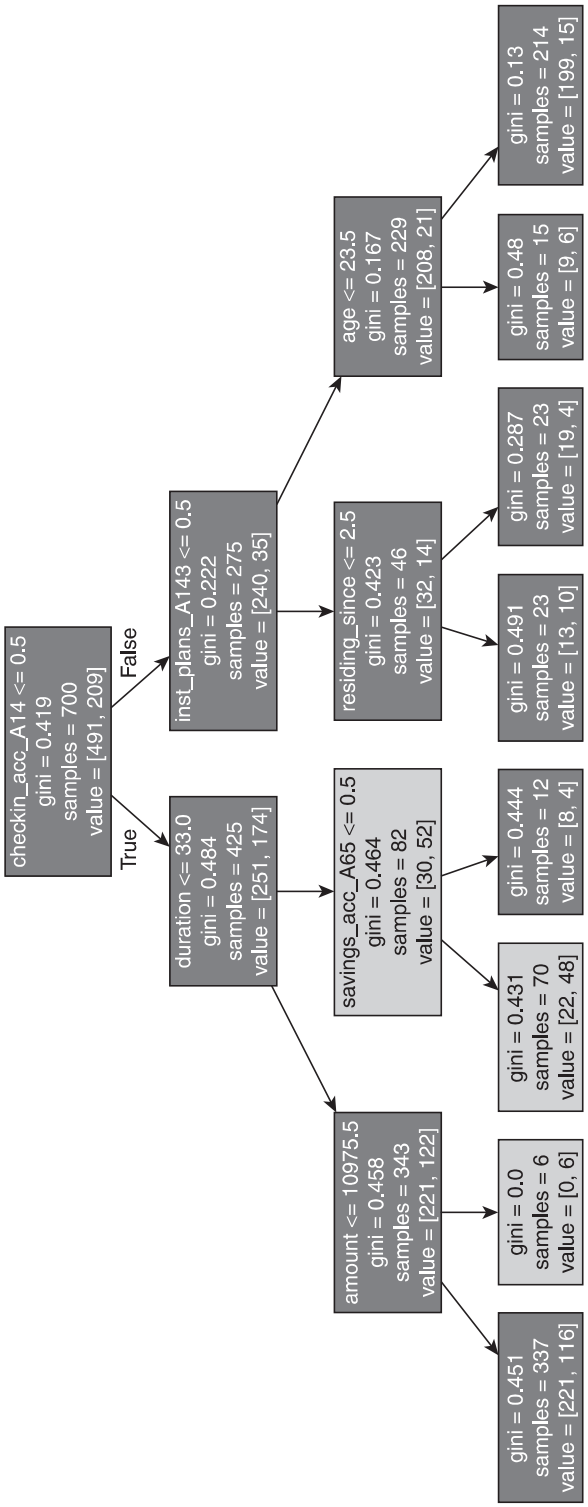


FIGURE 5.9 Decision tree graph created using Gini impurity splitting criteria.

So, in the top node the probability of finding good credit is 491/700 and finding bad credit is 209/700. Gini impurity for the top node in the above decision tree will be

```
gini_node_1 = 1 - pow(491/700, 2) - pow (209/700, 2)
print( round( gini_node_1, 4) )
```

```
0.419
```

The Gini index is 0.419 as shown in Figure 5.9.

5.5.6 | Building Decision Tree using Entropy Criteria

Entropy is another popular measure of impurity that is used in classification trees to split a node. Assume that there are J classes labelled 1, 2, …, J. The entropy at node k is given by

$$\text{Entropy}(k) = -\sum_{j=1}^{J} p(j|k)\log_2(j|k) \tag{5.9}$$

The value of entropy lies between 0 and 1, with a higher entropy indicating a higher impurity at the node. The following codes show how to build a classification tree using entropy splitting criteria and then displaying the tree classifier using Graphviz.

```
clf_tree_entropy = DecisionTreeClassifier( criterion = 'entropy',
                                           max_depth = 3 )
clf_tree_entropy.fit( X_train, y_train )

# Export the tree into odt file
export_graphviz( clf_tree_entropy,
                 out_file = "chd_tree_entropy.odt",
                 feature_names = X_train.columns )

# Read the create the image file
chd_tree_graph = pdot.graphviz.graph_from_dot_file( 'chd_tree_
entropy.odt' )
chd_tree_graph.write_jpg ( 'chd_tree_entropy.png' )
# Render the png file
Image(filename='chd_tree_entropy.png')
```

From the decision tree diagram in Figure 5.10, it can be interpreted that

1. The nodes represented by dark shades depict majority good credits, while the nodes represented by light shades are majority bad credits.
2. The rules generated at the top two level nodes are same as rules generated by decision tree built using Gini index. The rules at the third level nodes are different.
3. One of the rules can be interpreted as: If the customer has *checkin_account_A14*, *Inst_plans* code is A143 and credit history code is A34, then there is a high probability of being good credit. There are 96 customers who satisfy these criteria and 94 of them have good credit.

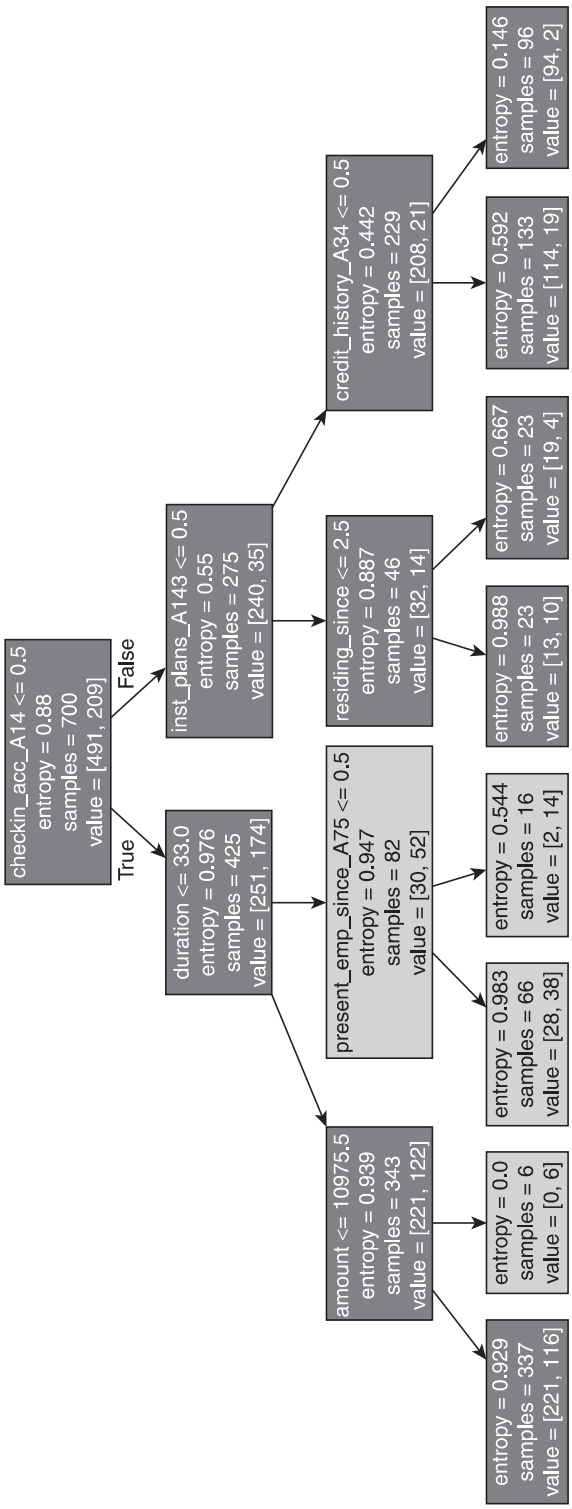


FIGURE 5.10 Decision tree graph created using entropy splitting criteria.

Calculating Entropy

As show in Figure 5.10, at the top node there are 700 observations of which 491 are good credits and 209 are bad credits. At the top of the node, the probability of finding good credit is 491 and not finding bad credit is 209. So, entropy at the node is given by [as per Eq. (5.9)]

```
import math

entropy_node_1 = - (491/700)*math.log2(491/700) - (209/700)*math.
                   log2(209/700)
print(round( entropy_node_1, 2))
```

```
0.88
```

The entropy at the top node is 0.88, as shown in the Figure 5.10.

Measuring Test Accuracy using AUC

```
tree_predict = clf_tree_entropy.predict( X_test )
metrics.roc_auc_score( y_test, tree_predict )
```

```
0.5763
```

5.5.7 | Finding Optimal Criteria and Max Depth

We need to search through various combinations of criteria and tree depths to find optimal criteria and depth where the tree gives highest accuracy.

sklearn.model_selection provides a feature called *GridSearchCV* which searches through a set of possible hyperparameter values and reports the most optimal one. *GridSearchCV* needs to be provided with the accuracy measure to be used for searching for the optimal value. For classification models, roc_auc values are mostly used. It uses *k*-fold cross validation to measure and validate the accuracy.

GridSearchCV can be used for any machine learning model and can search through multiple hyperparameters of the model. It takes the following arguments:

1. estimator – A scikit-learn model which implements the estimator interface.
2. param_grid – A dictionary with parameter names (string) as keys and lists of parameter settings to try as values.
3. scoring – Is a string; is the accuracy measure, i.e. *roc_auc*.
4. cv – Is an integer; gives the number of folds in *k*-fold. (More on *k*-fold cross-validation in *Chapter 6: Advanced Machine Learning*.)

Configuring the grid search to search for optimal parameters

1. Splitting criteria: gini or entropy.
2. Maximum depth of decision tree ranging from 2 to 10.

The searching of optimal parameter will be validated using 10-fold cross validation and the most optimal parameter will be chosen based on ROC AUC score.

```
from sklearn.model_selection import GridSearchCV
tuned_parameters = [{'criterion': ['gini','entropy'],
                     'max_depth': range(2,10)}]
clf_tree = DecisionTreeClassifier()
clf = GridSearchCV(clf_tree,
      tuned_parameters,
      cv=10,
      scoring='roc_auc')
clf.fit(X_train, y_train)
```

```
GridSearchCV(cv=10, error_score='raise',
       estimator=DecisionTreeClassifier(class_weight=None,
           criterion='gini', max_depth=None,
           max_features=None, max_leaf_nodes=None,
           min_impurity_decrease=0.0, min_impurity_split=None,
           min_samples_leaf=1, min_samples_split=2,
           min_weight_fraction_leaf=0.0,presort=False,random_state=None,
           splitter='best'),
       fit_params=None, iid=True, n_jobs=1,
       param_grid=[{'max_depth': range(2, 10), 'criterion': ['gini',
       'entropy']}],
       pre_dispatch='2*n_jobs', refit=True, return_train_score='warn',
       scoring='roc_auc', verbose=0)
```

Grid search returns *bestscore* and corresponding *bestparams* after searching through all combinations.

```
clf.best_score_
```

```
0.68242993197278912
```

```
clf.best_params_
```

```
{'criterion': 'gini', 'max_depth': 2}
```

The tree with *gini* criteria and *max_depth* = 2 is the best model. Finally, we can build a model with these parameters and measure the accuracy of the test. This is left to the readers for trying out. Refer to *Section 5.5.2: Building Decision Tree Classifier using Gini Criteria*.

5.5.8 | Benefits of Decision Tree

The benefits of building a decision tree classifier are as follows:

1. Rules generated are simple and interpretable. Trees can be visualized.
2. Work well with both numerical and categorical data. Do not require data to be normalized or creation of dummy variables.
3. Rules can help create business strategies.

CONCLUSION

1. In classification problem, the dependent variable *Y* takes finite discrete values. Logistic regression is one of the most popular techniques used for solving classification problems.
2. Logistic regression provides the probability of occurrence of the event; final class is usually predicted using a classification cut-off probability.
3. Decision trees generate rules that can be interpreted and business can create strategies around it.
4. The accuracy of an LR model is measured using metrics such as sensitivity, specificity, F-score, precision, and area under the ROC curve. Final model selection may be done using any of these metrics.
5. The optimal classification cut-off probability is usually calculated using Youden's Index or cost-based approach.
6. Gain and lift charts are two approaches used while solving classification problems with imbalanced datasets.

EXERCISES

Answer Questions 1 to 10 using the *SAheart* Dataset.

The dataset SAheart.data is taken from the link below

http://www-stat.stanford.edu/~tibs/ElemStatLearn/datasets/SAheart.data

The dataset contains records of males in a heart-disease high-risk region of the Western Cape, South Africa. There are roughly two controls per case of CHD. Many of the CHD positive men have undergone blood pressure reduction treatment and other programs to reduce their risk factors after their CHD event. In some cases, the measurements were made after these treatments. These data are taken from a larger dataset, described in Rousseauw et al. (1983), *South African Medical Journal*. It is a tab separated file (.csv) and contains the following columns:

- *sbp* – Systolic blood pressure
- *tobacco* – Cumulative tobacco (kg)
- *ldl* – Low densiity lipoprotein cholesterol
- *adiposity*
- *famhist* – Family history of heart disease (Present, Absent)
- *typea* – type-A behavior
- *obesity*
- *alcohol* – Current alcohol consumption
- *age* – Age at onset
- *chd* – Response, coronary heart disease

1. Build a logistic regression model to predict the probability of getting *chd* (Coronary Heart Disease), that is, $P(chd = 1)$. Use all other columns except *chd* as features to build the model.
2. Find out which features are statistically significant in the logistic regression model built in Question 1. Build a new logistic regression model using only the significant features.
3. From the logistic regression model parameters, determine which parameters affect the probability of *chd* positively and negatively.

4. Calculate Youden's index for all possible cut-off probabilities ranging from 0.1 to 0.5 with an interval of 0.01. Find the optimal cut-off where Youden's index is maximum.
5. Build a confusion matrix based on the cut-off probability found in Question 4 and report the precision and recall of the model for *chd* cases (i.e., *chd* = 1).
6. Find optimal cut-off probability using cost-based approach using the cost of FPs and FNs as defined below.
   - Cost of predicting "chd" as "No chd" (FNs) cases is 5 times more than predicting "No chd" as "chd" (FPs).
   - Then find the precision and recall of the model for *chd* = 1.
7. Build a decision tree classifier model to predict the probability of using Gini index and max depth as 4.
8. Compare the decision tree classifier and logistic regression model performance. Explain which accuracy score can be used to find out which model is better in classifying *chd* from *no chd* cases.
9. Plot the decision tree and find out the most important splitting criteria at the top node. Explain how the Gini index is derived at the top node of the decision tree classifier.
10. Find the optimal *max_depth* for decision tree classifier if Gini index is used as splitting criteria. Search possible depths ranging from 3 to 10 and use ROC and AUC as scoring to find the optimal *max_depth*.

Answer Questions 11 to 15 using the *HR* Dataset.

The dataset *hr_data.csv* contains samples of candidates that were part of a recruitment process of a particular client of ScaleneWorks. ScaleneWorks supports several information technology (IT) companies in India with their talent acquisition. One of the challenge they face is about 30% of the candidates who accept the jobs offer, do not join the company. This leads to huge loss of revenue and time as the companies initiate the recruitment process again to fill the workforce demand. ScaleneWorks wants to find out if a model can be built to predict the likelihood of a candidate joining the company. If the likelihood is high, then the company can go ahead and offer the jobs to the candidates.

The dataset contains several attributes about candidates along with a column (or variable) that indicates if the candidate finally joined the company or not.

Here is the description of candidate's attributes:

- *Candidate* – Reference number; it is a unique number to identify the candidate
- *DOJ extended* – Binary variable identifying whether candidate asked for date of joining extension (Yes/No)
- *Duration to accept the offer* – Number of days taken by the candidate to accept the offer (Scale variable)
- *Notice period* – Notice period to be served in the parting company before candidate can join this company (Scale variable)
- *Offered band* – Band offered to the candidate based on experience, performance in interview rounds (C0/C1/C2/C3/C4/C5/C6)
- *Percentage hike expected* – Percentage hike expected by the candidate (Scale variable)
- *Percentage hike offered* – Percentage hike offered by the company (Scale variable)
- *Joining bonus* – Binary variable indicating if joining bonus was given or not (Yes/no)
- *Gender* – Gender of the candidate (Male/Female)
- *Candidate source* – Source from which resume of the candidate was obtained (Employee referral/Agency/ Direct)

- *REX* (in Yrs.) – Relevant years of experience of the candidate for the position offered (Scale variable)
- *LOB* – Line of business for which offer was rolled out (Categorical variable)
- *Date of Birth* – Date of birth of the candidate
- *Joining location* – Company location for which the offer was rolled out for the candidate to join (Categorical variable)
- *Candidate relocation status* – Binary variable indicating whether the candidate has to relocate from one city to another city for joining (Yes/No)
- *HR Status* – Final joining status of the candidate (Joined/Not Joined)

11. Build a logistic regression model to predict the probability of a candidate joining the company. Assume "Not Joined" as positive cases and "Joined" as negative cases.
12. Find the significant features from the above model and build another logistic regression model with only the significant features.
13. Assume the following costs to find optimal cut-off probability to determine if a candidate will join or not.

- Cost of predicting "Not Joining" as "Joining" (FPs) cases is 3 times more than predicting "Joining" as "Not Joining" (FNs).

14. Build a confusion matrix based on the cut-off probability found in Question 13 and report the precision and recall of the model for joining cases.
15. HR wants to understand the key parameters effecting the joining of candidates. So, build a decision tree with optimal parameters and provide some rules to HR for building strategies to ensure candidates offered job most likely will join the company in future.

REFERENCES

1. U Dinesh Kumar (2017). *Business Analytics: The Science of Data-Driven Decision Making*, Wiley India, India.
2. UCI Machine Learning Repository: https://archive.ics.uci.edu/ml/
3. Youden W J (1950). Index for Rating Diagnostic Tests, *Cancer*, Vol. 3, No. 1, pp. 32–35.
4. Scikit-learn documentation: http://scikit- learn.org/
5. Statsmodel: https://www.statsmodels.org/stable/index.html
6. Pandas Library: https://pandas.pydata.org/
7. Matplotlib Library: https://matplotlib.org/
8. Seabon Library: https://seaborn.pydata.org/
9. Rousseauw J, du Plessis J, Benade A, Jordaan P, Kotze J, and Ferreira J (1983). Coronary Risk Factor Screening in Three Rural Communities, *South African Medical Journal*, Vol. 64, pp. 430–436.

Chapter 05_Classification.indd 178 4/24/2019 6:55:20 PM